

2-2 - SQL trigger syntax



give me syntax for a sql trigger

Certainly! Here is the basic syntax for creating a **trigger** in SQL (most commonly used in MySQL, PostgreSQL, SQL Server, etc.). The example below is written in MySQL-style syntax, which is the most widely used and easily adaptable.

Basic SQL Trigger Syntax (MySQL Example):

```
sql

CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- Triggered SQL statements go here
END;
```

Breakdown:

- **CREATE TRIGGER trigger_name:** Give your trigger a unique name.
- **{BEFORE | AFTER}:** Specify when the trigger should activate (before or after the event).
- **{INSERT | UPDATE | DELETE}:** The event that will fire the trigger.
- **ON table_name:** The table the trigger is associated with.
- **FOR EACH ROW:** The trigger fires for each affected row (row-level trigger).
- The **triggered code** (in the **BEGIN...END;** block) is the SQL you want to run when the trigger fires.

Example:

Suppose you want to create a trigger that automatically updates an **updated_at** column every time a row in the **users** table is updated:

```
sql

CREATE TRIGGER before_user_update
BEFORE UPDATE ON users
FOR EACH ROW
BEGIN
    SET NEW.updated_at = NOW();
END;
```

Note for SQL Server (T-SQL):

SQL Server's trigger syntax is a bit different:

```
sql

CREATE TRIGGER trigger_name
ON table_name
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    -- Triggered SQL statements go here
END;
```

If you tell me your **DBMS** (MySQL, PostgreSQL, SQL Server, Oracle, etc.), I can give you a syntax tailored for that system, along with a practical example! Let me know if you want that.



give me syntax for oracle

Absolutely! Here's the **Oracle SQL trigger syntax** (Oracle PL/SQL):

Basic Trigger Syntax in Oracle

```
sql

CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
```

```
{INSERT | UPDATE | DELETE}
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    -- Variable declarations (optional)
BEGIN
    -- PL/SQL statements (trigger logic)
END;
/
```

Explanation:

- **CREATE [OR REPLACE] TRIGGER trigger_name**: Creates a new trigger or replaces an existing one.
- **{BEFORE | AFTER | INSTEAD OF}**: When to fire the trigger (before or after the triggering event).
- **{INSERT | UPDATE | DELETE}**: Triggering event(s). You can use multiple events: **INSERT OR UPDATE OR DELETE**.
- **ON table_name**: The table the trigger is associated with.
- **[FOR EACH ROW]**: Row-level trigger (fires for each row affected). Omit for statement-level triggers.
- **[WHEN (condition)]**: Optional, adds a condition to fire the trigger.
- **DECLARE ...**: Optional variable declarations.
- **BEGIN ... END;**: The logic to execute.
- **/**: Used in SQL*Plus and many IDEs to end the PL/SQL block.

Example:

Suppose you want a trigger that sets the **updated_at** column to the current timestamp **before** any update on the **users** table:

```
sql

CREATE OR REPLACE TRIGGER set_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
BEGIN
    :NEW.updated_at := SYSDATE;
```

END;
/

Key Oracle trigger points:

- Use `:NEW.column_name` for new values (in INSERT/UPDATE triggers).
- Use `:OLD.column_name` for old values (in UPDATE/DELETE triggers).

If you want a specific example (e.g., after insert, multiple events, etc.), just ask!



9.1.3 Event Attribute Functions

These are system-defined event attribute functions. For instance:

```
ORA_DICT_OBJ_NAME
ORA_DICT_OBJ_OWNER
ORA_DICT_OBJ_TYPE
ORA_CLIENT_IP_ADDRESS
ORA_DATABASE_NAME
ORA_TS_ALTER_COLUMN
ORA_LOGIN_USER
SPACE_ERROR_INFO
ORA_SYSEVENT
```

How to use them: Just like any other function.

Example:

```
DECLARE
password VARCHAR2(40);
BEGIN
IF ora_dict_obj_type = 'USER' THEN
password := ora_ora_encrypted_password;
END IF;
END;

DECLARE
ip_address VARCHAR2(11);
BEGIN
IF ora_sysevent = 'LOGIN' THEN
ip_address := ora_client_ip_address;
END IF;
END;
```

66

explain this part

Sure! Let me explain **section 9.1.3 "Event Attribute Functions"** and the code examples from your Oracle documentation:

What are Event Attribute Functions?

- These are **built-in Oracle functions** that provide information about the context of a database event (like a trigger firing, a user logging in, an object being modified, etc.).
- You can use these functions **inside triggers or event handling code** to fetch details about what is happening, who is doing it, and on what object.

List of Example Event Attribute Functions:

- **ORA_DICT_OBJ_NAME** – Name of the object involved in the event (e.g., table, user).
 - **ORA_DICT_OBJ_OWNER** – Owner of the object.
 - **ORA_DICT_OBJ_TYPE** – Type of object (TABLE, USER, etc.).
 - **ORA_CLIENT_IP_ADDRESS** – Client's IP address (useful in logon/logoff triggers).
 - **ORA_DATABASE_NAME** – Name of the database where the event occurred.
 - **ORA_IS_ALTER_COLUMN** – Indicates if the event is an ALTER COLUMN operation.
 - **ORA_LOGIN_USER** – User who is currently logged in.
 - **SPACE_ERROR_INFO** – Information about space errors (e.g., running out of disk).
 - **ORA_SYSEVENT** – Type of system event that fired the trigger (e.g., 'LOGON', 'LOGOFF', 'STARTUP').
-

How to Use Them?

- You use these **just like regular functions** inside your PL/SQL code (triggers, procedures).
 - They help you make decisions or log information **based on the context of the event**.
-

Example 1 Explained:

```
sql

DECLARE
  password VARCHAR2(60);
BEGIN
  IF ora_dict_obj_type = 'USER' THEN
    password := ora_des_encrypted_password;
  END IF;
END;
```

- Here, you are checking **what type of object** is involved in the event (using `ora_dict_obj_type`).

- If the object is a **USER**, you get the encrypted password (`ora_des_encrypted_password`) and assign it to the `password` variable.
- This might be used in a **trigger on USER creation or modification**, for example, to audit or capture passwords.

Example 2 Explained:

```
sql

DECLARE
  ip_address VARCHAR2(11);
BEGIN
  IF ora_sysevent = 'LOGON' THEN
    ip_address := ora_client_ip_address;
  END IF;
END;
```

- Here, you are checking if the system event (`ora_sysevent`) is 'LOGON' (i.e., a user is logging into the database).
- If so, you capture the user's **IP address** (`ora_client_ip_address`).
- This could be used in a **LOGON trigger** to record login details for security/auditing.

In Summary:

- These attribute functions give you **real-time context** during triggers/events.
- They let you write **intelligent triggers** that react to the specific event, user, object, or even client IP address.

If you want to see how these are used in a **real trigger**, just let me know!



Few Points:

- **Schema trigger:** The keyword schema decides its type. A SCHEMA trigger is created on a schema and fires whenever the user who owns it is the current user and initiates the triggering event.

Suppose that both user1 and user2 own schema triggers, and user1 invokes a DR unit owned by user2. Inside the DR unit, user2 is the current user. Therefore, if the DR unit initiates the triggering event of a schema trigger that user2 owns, then that trigger fires. However, if the DR unit initiates the triggering event of a schema trigger that user1 owns, then that trigger does not fire. [present a suitable example in this regard.]

- **DATABASE Triggers:** Use keyword DATABASE for it. A DATABASE trigger is created on the database and fires whenever any database user initiates the triggering event.

Example Suppose we want to log 3 types of DDL (CREATE, ALTER AND DELETE) for a particular user (schema).

Note: Event Attribute Functions have been used to get the necessary information.

explain this

Absolutely! Let's break down and explain each part from your text about **Schema triggers** and **Database triggers** in Oracle:

Schema Trigger

- **Definition:**

A **Schema trigger** is a trigger that is attached to a specific schema (i.e., a user account in Oracle).

- **When does it fire?**

It fires whenever the **owner of the trigger** (the user who created it) is the one executing an event (like a DDL statement—CREATE, ALTER, DROP, etc.) **within their own schema**.

Detailed Example from the Text:

Suppose you have two users:

- **user1** (owns a schema, has a schema trigger)
- **user2** (owns a schema, has a schema trigger)

Now, if:

- **user1** initiates an event (e.g., runs a DDL statement) **inside user2's schema**, user2 is considered the "current user" for that event.
 - So, the **schema trigger owned by user2** will fire (because the action is in user2's schema).

- The **schema trigger owned by user1** will **not fire** for this event.
- If the action is performed **inside user1's schema**, only user1's schema trigger will fire.

In summary:

Schema triggers fire only for events initiated by their owner in their own schema.

Database Trigger

- **Definition:**
A **Database trigger** uses the **DATABASE** keyword and is created at the **whole database** level (not just a single schema).
 - **When does it fire?**
It fires whenever **any user** in the database initiates the triggering event (such as a DDL statement), **regardless of which schema is involved**.
-

Example Use Case (from your text):

Suppose we want to log 3 types of DDL (CREATE, ALTER, and DELETE) for a particular user (schema).

- You could use **event attribute functions** (from your previous question) in these triggers to capture details about what DDL statement was run, by whom, and on what object.
-

Key Differences:

- **Schema Trigger:**
 - Linked to a **specific schema (user)**.
 - Fires **only when that schema's owner** performs a relevant action **in their own schema**.
 - **Database Trigger:**
 - Linked to the **whole database**.
 - Fires for **any user** performing a relevant action, anywhere in the database.
-

Summary Table:

Trigger Type	Scope	Who Fires It	Typical Use
Schema Trigger	One Schema	That schema's owner	Logging/auditing per user
Database Trigger	Whole Database	Any user in the database	Enforcing organization-wide rules

If you want, I can give you **example code** of both types of triggers! Let me know if you want to see how they're created.



give me the difference between Schema Trigger and Database trigger with understandable examples

Of course! Here's a **simple comparison** between Schema Trigger and Database Trigger in Oracle, with easy-to-understand examples for each.

Difference Between Schema Trigger and Database Trigger

Feature	Schema Trigger	Database Trigger
Scope	Specific to one user/schema	Entire database (all users/schemas)
Who can fire?	Only the trigger owner, within their own schema	Any user in the database
Typical Use	Per-user or per-schema auditing or logic	Enforce or log actions for all users

Schema Trigger

Definition:

A trigger that fires for DDL (CREATE, ALTER, DROP, etc.) or logon/logoff events **performed by the schema owner** in their own schema.

Example:

Let's say you want to log whenever the user **ALICE** creates or drops a table in her own schema.

```
sql

CREATE OR REPLACE TRIGGER log_alice_ddl
AFTER CREATE OR DROP ON SCHEMA
BEGIN
    INSERT INTO ddl_audit_log (username, event_type, event_date)
    VALUES (USER, ORA_SYSEVENT, SYSDATE);
END;
/
```

- This trigger will **only fire when Alice herself** creates or drops a table in her schema.
- If another user (BOB) creates/drops a table in their schema, **this trigger does not fire**.

Database Trigger

Definition:

A trigger that fires for certain events **performed by any user, anywhere in the database**.

Example:

Suppose you want to log **every CREATE TABLE** statement executed by any user in the database.

```
sql

CREATE OR REPLACE TRIGGER log_all_ddl
AFTER CREATE ON DATABASE
BEGIN
    INSERT INTO ddl_audit_log (username, event_type, object_name, event_date)
    VALUES (ORA_LOGGIN_USER, ORA_SYSEVENT, ORA_DICT_OBJ_NAME, SYSDATE);
END;
/
```

- This trigger will **fire for any user** who creates a table, in any schema, anywhere in the database.

Quick Summary Table

Scenario	Fires Schema Trigger?	Fires Database Trigger?
Alice creates a table in Alice's schema	Yes	Yes
Bob creates a table in Bob's schema	No (for Alice)	Yes
Alice creates a table in Bob's schema (if permitted)	No	Yes

In Simple Terms:

- **Schema Trigger:** "Watch my own house. I care only what I do in my place."
- **Database Trigger:** "Watch the whole building. Whoever does anything, I want to know!"

Let me know if you want to see a more detailed code example or an explanation of the logging tables!



```
create or replace trigger ddl_trigger
before create or alter or drop on SCHEMA
begin
  dbms_output.put_line('Who did it? ' || ora_dict_obj_owner);
  dbms_output.put_line('What was the Operation? ' || ora_sysevent);
  dbms_output.put_line('On what? ' || ora_dict_obj_name);
  dbms_output.put_line('On type of object it was? ' || ora_dict_obj_type);
end;
```

fully explain this code

Certainly! Let's break down and fully explain your trigger code, **line by line**, so you understand exactly what's happening and why.

Code Breakdown

sql

```
create or replace trigger ddl_trigger
before create or alter or drop on SCHEMA
begin
  dbms_output.put_line('Who did it? ' || ora_dict_obj_owner);
  dbms_output.put_line('What was the Operation? ' || ora_sysevent);
  dbms_output.put_line('On what? ' || ora_dict_obj_name);
```

```
dbms_output.put_line('On type of object it was? '||ora_dict_obj_type);  
end;
```

1. create or replace trigger ddl_trigger

- **Creates a new trigger** called `ddl_trigger`.
- If a trigger with this name already exists, it will be replaced.

2. before create or alter or drop on SCHEMA

- This trigger fires **before** any **CREATE**, **ALTER**, or **DROP** statement is executed on **any object in the current schema**.
- In other words, any time you try to create, change, or remove a database object (table, view, etc.) in your schema, this trigger will run **before** the action takes place.

3. begin ... end;

- This section contains the **PL/SQL code block** that is executed when the trigger fires.

Inside the Trigger Block

Here, four lines use `dbms_output.put_line` to print information. Let's explain each:

a. `dbms_output.put_line('Who did it? '||ora_dict_obj_owner);`

- `dbms_output.put_line`: This function prints text to the SQL*Plus or SQL Developer output window.
- `ora_dict_obj_owner`: This built-in Oracle event attribute function gives the **owner of the object** being affected by the DDL event (usually the schema/user).
- **Result:**
Prints who owns the object on which the DDL is being executed.

b. `dbms_output.put_line('What was the Operation? '||ora_sysevent);`

- **ora_sysevent:** Returns the **type of DDL event** that caused the trigger to fire (CREATE, ALTER, or DROP).
- **Result:**
Prints the operation being performed.

c. `dbms_output.put_line('On what? '||ora_dict_obj_name);`

- **ora_dict_obj_name:** Returns the **name of the object** being created, altered, or dropped (e.g., the table or view name).
- **Result:**
Prints which object is affected.

d. `dbms_output.put_line('On type of object it was? '||ora_dict_obj_type);`

- **ora_dict_obj_type:** Returns the **type of the object** (such as TABLE, VIEW, PROCEDURE, etc.).
- **Result:**
Prints what kind of object the event affected.

Summary of What the Trigger Does:

Whenever you try to create, alter, or drop an object in your schema:

- **Before** the operation, this trigger prints:
 - **Who** owns the object (user/schema)
 - **What** operation is being performed (CREATE, ALTER, DROP)
 - **Which object** is affected (object name)
 - **What type** of object is involved (e.g., TABLE, VIEW)

Purpose:

This is mostly used for **auditing** or **debugging**—to see what DDL actions are happening, by whom, and on what.

Example Output

Suppose user **ALICE** tries to drop a table named **EMPLOYEES** in her schema. The output would look like:

```
pgsql
```

```
Who did it? ALICE
What was the Operation? DROP
On what? EMPLOYEES
On type of object it was? TABLE
```

In Short:

- **This trigger helps you monitor and log schema changes in your database,** making it easier to track what is being created, altered, or dropped, and by whom, in real time.

Let me know if you want to see how to save this output to a table (for a real audit log), or if you want more trigger examples!



now give me the differences between DDL trigger and DML trigger

Absolutely! Here's a clear and structured comparison between **DDL triggers** and **DML triggers** in Oracle (or any relational database that supports triggers):

Difference Between DDL Trigger and DML Trigger

Aspect	DDL Trigger	DML Trigger
Full Form	Data Definition Language Trigger	Data Manipulation Language Trigger
Fires On	DDL events: CREATE, ALTER, DROP, TRUNCATE, RENAME, etc.	DML events: INSERT, UPDATE, DELETE
Purpose	Monitor, restrict, or log structural/schema changes	Monitor, restrict, or process data changes
Affects	Database objects (tables, views, users, etc.)	Table data (rows in tables)
Typical Use	Auditing, restricting DDL changes, logging schema activity	Enforcing business rules, auditing data changes, validation

Aspect	DDL Trigger	DML Trigger
Access to :NEW, :OLD	Not available (DDL triggers don't see row data)	Available (for accessing values before/after changes)
Event Attribute Functions	Use functions like ORA_SYSEVENT , ORA_DICT_OBJ_NAME	Use pseudorecords like :NEW and :OLD

Examples

1. DDL Trigger Example

Purpose: Log whenever a table is created or dropped.

```
sql

CREATE OR REPLACE TRIGGER log_table_ddl
AFTER CREATE OR DROP ON DATABASE
BEGIN
    INSERT INTO ddl_audit_log (event, object_name, event_time)
    VALUES (ora_sysevent, ora_dict_obj_name, SYSDATE);
END;
/
```

- **Fires:** When any table is created or dropped in the database.
- **Cannot access** row data.

2. DML Trigger Example

Purpose: Automatically set an **updated_at** column whenever a row is updated.

```
sql

CREATE OR REPLACE TRIGGER set_updated_at
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    :NEW.updated_at := SYSDATE;
END;
/
```

- **Fires:** Each time a row in the **employees** table is updated.
- **Can access** **:NEW** and **:OLD** values for row-level changes.

Quick Summary Table

Feature	DDL Trigger	DML Trigger
Works on	Schema/object changes	Table data changes
Row access	No (:NEW, :OLD not available)	Yes (:NEW, :OLD available)
Common uses	Audit DDL, restrict schema changes	Audit DML, enforce data consistency

Simple Analogy

- **DDL Trigger:** "Alert me whenever someone builds, demolishes, or remodels a room in the building."
- **DML Trigger:** "Alert me whenever someone changes something inside a room (like adding/removing/changing furniture)."

If you want **more details or example code**, just let me know!



9.1.5 Data Manipulation Language Triggers

DML triggers can fire before or after INSERT, UPDATE, and DELETE statements.

Row-level and statement-level trigger: DML triggers can be statement- or row-level activities. Statement-level triggers fire and perform a statement or set of statements once no matter how many rows are affected by the DML event. Row-level triggers fire and perform a statement or set of statements for each row changed by a DML statement.

DML triggers Syntax:

```
CREATE [OR REPLACE] TRIGGER trigger_name
(BEFORE | AFTER)
(INSERT | UPDATE | DELETE OF column [, column2 [, column(n+1)]] | DELETE
ON table_name
[FOR EACH ROW])
[WHEN (logical_expression)]
[DECLARE]
[PRAGMA AUTONOMOUS_TRANSACTION];
declaration_statements;
BEGIN
statement_statements;
END trigger_name;
```

Note: The clause *[FOR EACH ROW]* specifies that it is row-level trigger.

Example:

```
CREATE OR REPLACE TRIGGER demo_trigger_type1
BEFORE DELETE OR INSERT OR UPDATE ON employees
--FOR EACH ROW

--DECLARE

BEGIN

dbms_output.put_line('Row Level Trigger Fires each time. ');

END;

CREATE OR REPLACE TRIGGER demo_trigger_type1
BEFORE DELETE OR INSERT OR UPDATE ON employees
```

68

Dr. Raihan Kamal

CHAPTER 9. DATABASE TRIGGERS

```
--FOR EACH ROW

--DECLARE

BEGIN

dbms_output.put_line('Row Level Trigger Fires - ');

END;
```

explain DML trigger syntax

Sure! Let's break down the **DML trigger syntax** from your screenshot and explain each part for clarity.

What is a DML Trigger?

- **DML triggers** are database triggers that fire **before or after INSERT, UPDATE, or DELETE** operations on a table.
- They are used to enforce business rules, validate data, automate actions, audit changes, etc.

Syntax Breakdown

```
sql

CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT | UPDATE | DELETE [OF column1 [, columnN]]}
ON table_name
[FOR EACH ROW]
[WHEN (logical_expression)]
[DECLARE
  [PRAGMA AUTONOMOUS_TRANSACTION;]
  declaration_statements;
]
BEGIN
  execution_statements;
END [trigger_name];
```

Explanation of Each Clause

1. CREATE [OR REPLACE] TRIGGER trigger_name

- Create a new trigger (or replace it if it already exists).
- **trigger_name** is the name you give the trigger.

2. {BEFORE | AFTER}

- Decide whether the trigger fires **before** or **after** the DML event (INSERT, UPDATE, DELETE).

3. {INSERT | UPDATE | DELETE [OF column1 [, columnN]]}

- The DML event(s) that will fire the trigger.
- You can specify multiple events (e.g., **BEFORE INSERT OR UPDATE**).
- For UPDATE, you can specify columns (e.g., **UPDATE OF salary**).

4. **ON table_name**

- The table the trigger is attached to.

5. **[FOR EACH ROW]** (optional)

- If present, this makes the trigger a **row-level trigger** (fires once per affected row).
- If omitted, it's a **statement-level trigger** (fires once per statement, no matter how many rows are affected).

6. **[WHEN (logical_expression)]** (optional)

- A condition that must be true for the trigger to fire (only valid for row-level triggers).

7. **[DECLARE ...]** (optional)

- Section for declaring variables, cursors, or marking the trigger as autonomous.

8. **BEGIN ... END;**

- The block of code (PL/SQL) that is executed when the trigger fires.
- Here, you write the logic you want to run.

Example from the Image Explained

sql

```
CREATE OR REPLACE TRIGGER demo_trigger_types1
BEFORE DELETE OR INSERT OR UPDATE ON employees
FOR EACH ROW

BEGIN
    dbms_output.put_line('Row Level Trigger Fires each time.');
```

```
END;
```

Explanation:

- The trigger is named **demo_trigger_types1**.
- It will fire **before** any **DELETE**, **INSERT**, or **UPDATE** operation on the **employees** table.

- It is a **row-level trigger** (**FOR EACH ROW**), so it fires **once for each affected row**.
- For each row changed, it prints a message using `dbms_output.put_line`.

Key Points:

- **Row-level** triggers can use `:NEW` and `:OLD` to access new and old row values.
- **Statement-level** triggers (no **FOR EACH ROW**) cannot access individual row data.
- Use **WHEN** clause to further filter when the trigger should execute.

Summary Table

Part	Purpose
BEFORE/AFTER	When trigger fires (before or after DML)
INSERT/UPDATE/DELETE	What DML event fires the trigger
ON table_name	Table the trigger is attached to
FOR EACH ROW	Fires per row (row-level trigger)
WHEN	Only fires if this condition is true
DECLARE	Declare variables/cursors if needed
BEGIN ... END	The logic to run when trigger fires

If you want a real-world example (like logging salary changes, enforcing rules, etc.), just ask!



explain DDL trigger syntax in the same way

Of course! Here's a **clear breakdown of DDL trigger syntax** in Oracle, similar to what I did for DML triggers:

What is a DDL Trigger?

- **DDL triggers** fire on Data Definition Language events, such as **CREATE**, **ALTER**, **DROP**, **TRUNCATE**, or **RENAME** of database objects (tables, views, users, etc.).
- Used for auditing, restricting schema changes, enforcing naming conventions, etc.

DDL Trigger Syntax

```
sql

CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER}
{DDL_event1 [OR DDL_event2 ...]}
ON {DATABASE | SCHEMA}
[DECLARE
  -- variable declarations (optional)
]
BEGIN
  -- code to execute when trigger fires
END;
```

Explanation of Each Clause

Clause/Part	Meaning
CREATE [OR REPLACE] TRIGGER trigger_name	Creates or replaces a trigger with the name you specify
{BEFORE AFTER}	{BEFORE AFTER}
{DDL_event1 [OR DDL_event2 ...]}	The DDL operation(s) that will fire the trigger: e.g., CREATE , ALTER , DROP , RENAME , etc.
ON {DATABASE SCHEMA}	{DATABASE SCHEMA}
	- DATABASE : Fires for the whole database, for all users/events
	- SCHEMA : Fires only for the specific schema (user)
[DECLARE ...]	Optional: section to declare PL/SQL variables, cursors, etc.
BEGIN ... END;	The block of code (PL/SQL) that runs when the trigger fires

Example Syntax:

A. Database-level DDL Trigger:

```
sql

CREATE OR REPLACE TRIGGER ddl_log_db
AFTER CREATE OR DROP OR ALTER ON DATABASE
BEGIN
    -- Code here, e.g., write to audit table or print info
    dbms_output.put_line('A DDL event happened in the database.');
```

B. Schema-level DDL Trigger:

```
sql

CREATE OR REPLACE TRIGGER ddl_log_schema
BEFORE DROP OR ALTER ON SCHEMA
BEGIN
    -- Code here, e.g., restrict or log schema changes
    dbms_output.put_line('A DDL event happened in this schema.');
```

Event Attribute Functions (Common in DDL Triggers)

- **ORA_SYSEVENT:** The DDL event type (CREATE, ALTER, DROP, etc.)
- **ORA_DICT_OBJ_NAME:** Name of the object being affected
- **ORA_DICT_OBJ_TYPE:** Object type (TABLE, VIEW, etc.)
- **ORA_DICT_OBJ_OWNER:** Owner of the object

Example (in the trigger body):

```
sql

dbms_output.put_line('Event: ' || ora_sysevent);
dbms_output.put_line('Object: ' || ora_dict_obj_name);
dbms_output.put_line('Type: ' || ora_dict_obj_type);
```

Summary Table

Part	Purpose/Description
BEFORE/AFTER	When trigger fires (before or after the DDL event)

Part	Purpose/Description
CREATE/ALTER/DROP	Which DDL events fire the trigger
ON DATABASE/SCHEMA	Scope: entire database or only current schema (user)
DECLARE	(Optional) Declare variables
BEGIN ... END	The logic to run when the DDL event occurs

Key Points

- **DDL triggers** do not have access to row data (no `:NEW/:OLD`).
- They use event attribute functions to get details about the DDL event and affected object.
- Useful for database-wide or schema-wide auditing, restriction, or automation on DDL changes.

If you want a specific **real-world example** (like blocking table drops or logging all DDL actions), let me know!



9.1.6 OLD and NEW Pseudorecords

When a row-level trigger fires, the PL/SQL runtime system creates and populates the two pseudorecords OLD and NEW. They are called pseudorecords because they have some, but not all, of the properties of records.

For the row that the trigger is processing:

- For an INSERT trigger, OLD contains no values, and NEW contains the new values.
- For an UPDATE trigger, OLD contains the old values, and NEW contains the new values.
- For a DELETE trigger, OLD contains the old values, and NEW contains no values.

fully explain this

Certainly! Here's a **full explanation** of the concept of **OLD and NEW pseudorecords** in Oracle row-level triggers, as described in your image:

What are OLD and NEW Pseudorecords?

When you create a **row-level trigger** in Oracle (for INSERT, UPDATE, or DELETE on a table), Oracle gives you access to two special records for each affected row:

- **:OLD** — Holds the **original (old) values** of the row before the DML operation.
- **:NEW** — Holds the **new values** that the row will have after the DML operation.

They are called **pseudorecords** because they behave a lot like PL/SQL records (you can refer to individual columns using dot notation, like **:OLD.salary** or **:NEW.salary**), but they are not true records.

How are OLD and NEW used in Triggers?

Their availability depends on the type of DML event:

1. INSERT Trigger

- **:OLD** — Has **no values** (because the row didn't exist before).
- **:NEW** — Contains the **new values** being inserted into the table.

Example:

When a new employee is added:

- **:NEW.name** contains the new employee's name.
- **:OLD.name** does **not** exist (NULL).

2. UPDATE Trigger

- **:OLD** — Contains the **old values** (before the update).
- **:NEW** — Contains the **new values** (after the update, or what's being written).

Example:

When an employee's salary is updated:

- **:OLD.salary** is the previous salary.
- **:NEW.salary** is the updated salary.

3. DELETE Trigger

- **:OLD** — Contains the **old values** (the row that's about to be deleted).

- **:NEW** — Has **no values** (because the row will not exist after the delete).

Example:

When an employee is removed:

- **:OLD.emp_id** is the employee's ID being deleted.
- **:NEW.emp_id** does **not** exist (NULL).

Quick Reference Table

Operation	:OLD contains	:NEW contains
INSERT	No values (NULL)	New values
UPDATE	Old values	New values
DELETE	Old values	No values (NULL)

Why is this useful?

- You can **compare old and new values** to detect changes (e.g., log if salary increased, prevent unwanted updates, etc.).
- You can **enforce business rules** (e.g., prevent someone from reducing their own salary).
- You can **audit** what data was deleted, updated, or inserted.

Example Usage

sql

```
CREATE OR REPLACE TRIGGER check_salary_change
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary < :OLD.salary THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be decreased!');
    END IF;
END;
```

- Here, the trigger checks the old and new salary values for each row being updated.

Summary

- **:OLD and :NEW** pseudorecords let you see the values **before and after** a DML event for each row.
- They are essential for **row-level triggers** to make decisions based on what is changing in your data.

Let me know if you want more code examples or specific use cases!



```
CREATE OR REPLACE TRIGGER EVAL_CHANGE_TRIGGER
AFTER INSERT OR UPDATE OR DELETE
ON EVALUATIONS
DECLARE
log_action EVALUATIONS_LOG.action%TYPE;
BEGIN
IF INSERTING THEN
log_action := 'Insert';
ELSIF UPDATING THEN
log_action := 'Update';
ELSIF DELETING THEN
log_action := 'Delete';
ELSE
DBMS_OUTPUT.PUT_LINE('This code is not reachable.');
```

```
END IF;

INSERT INTO EVALUATIONS_LOG (log_date, action)
VALUES (SYSDATE, log_action);
END;
```

explain this

Absolutely! Here's a full explanation of the **EVAL_CHANGE_TRIGGER** trigger in your screenshot:

What Does This Trigger Do?

- This is an **AFTER** trigger on the **EVALUATIONS** table.
- It fires **after any row is INSERTED, UPDATED, or DELETED**.
- Its purpose is to **log what kind of change happened** (Insert, Update, or Delete) into a log table called **EVALUATIONS_LOG** along with the current date and time.

Line-by-Line Explanation

sql

```
CREATE OR REPLACE TRIGGER EVAL_CHANGE_TRIGGER  
AFTER INSERT OR UPDATE OR DELETE  
ON EVALUATIONS
```

- **Creates or replaces** a trigger named **EVAL_CHANGE_TRIGGER**.
- It fires **after** an **INSERT**, **UPDATE**, or **DELETE** operation **on the EVALUATIONS table**.

sql

```
DECLARE  
log_action EVALUATIONS_LOG.action%TYPE;
```

- **Declares a variable** called **log_action** whose data type matches the **action** column in the **EVALUATIONS_LOG** table (using **%TYPE**).

sql

```
BEGIN  
IF INSERTING THEN  
    log_action := 'Insert';  
ELSIF UPDATING THEN  
    log_action := 'Update';  
ELSIF DELETING THEN  
    log_action := 'Delete';  
ELSE  
    DBMS_OUTPUT.PUT_LINE('This code is not reachable.');
```

- ```
END IF;
```
- The **trigger logic starts**.
  - Oracle provides the special Boolean values **INSERTING**, **UPDATING**, and **DELETING** **within a trigger** to tell you which operation caused the trigger to fire.
    - If it's an insert, set **log\_action** to **'Insert'**.
    - If it's an update, set **log\_action** to **'Update'**.
    - If it's a delete, set **log\_action** to **'Delete'**.
  - The **ELSE** block shouldn't ever be reached (since only these three actions can fire this trigger), so it prints a debug message.

```
sql
```

```
INSERT INTO EVALUATIONS_LOG (log_date, action)
VALUES (SYSDATE, log_action);
END;
```

- **Inserts a record** into the **EVALUATIONS\_LOG** table.
  - **log\_date**: the current date and time (**SYSDATE**).
  - **action**: the type of operation performed (**Insert**, **Update**, or **Delete**).
- **Ends the trigger block.**

## How Does This Work?

Whenever a row in the **EVALUATIONS** table is inserted, updated, or deleted:

- The trigger fires.
- It detects the type of operation.
- It logs this operation (with a timestamp) in the **EVALUATIONS\_LOG** table.

## Example:

If someone runs:

```
sql
```

```
UPDATE EVALUATIONS SET score = 90 WHERE student_id = 1001;
```

- The trigger detects it's an **update** (**UPDATING** is true).
- It inserts a row into **EVALUATIONS\_LOG**:
  - **log\_date** = the current time,
  - **action** = 'Update'.

## Summary Table

| Event on EVALUATIONS | What is logged in EVALUATIONS_LOG?    |
|----------------------|---------------------------------------|
| INSERT               | log_date = SYSDATE, action = 'Insert' |
| UPDATE               | log_date = SYSDATE, action = 'Update' |
| DELETE               | log_date = SYSDATE, action = 'Delete' |

## Why is this useful?

- **Auditing:** Tracks all changes made to important data.
- **Debugging:** Helps see what operations are happening and when.
- **Security:** Can catch unauthorized or suspicious data changes.

Let me know if you want a visual diagram, more code examples, or explanation about trigger limitations!



### 9.1.7.1 Order of Firing Trigger

Oracle allows more than one trigger to be created *for the same timing point*, but it has *never guaranteed the execution order of those triggers*. The Oracle 11g trigger syntax now includes the *FOLLOWS* clause to guarantee execution order for triggers defined with the same timing point. The following example creates a table with two triggers for the same timing point.

```
CREATE TABLE trigger_follows_test (
 id NUMBER,
 description VARCHAR2(50)
);

CREATE OR REPLACE TRIGGER trigger_follows_test_trg_1
BEFORE INSERT ON trigger_follows_test
FOR EACH ROW
BEGIN
 DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_1 - Executed');
END;
/

CREATE OR REPLACE TRIGGER trigger_follows_test_trg_2
BEFORE INSERT ON trigger_follows_test
FOR EACH ROW
BEGIN
 DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_2 - Executed');
END;
/
```

If we insert into the test table, there is no guarantee of the execution order.

```
SQL> SET SERVEROUTPUT ON
SQL> INSERT INTO trigger_follows_test VALUES (1, 'ONE');
```

72

Dr. Raihan Kamal

CHAPTER 9. DATABASE TRIGGERS

```
TRIGGER_FOLLOWS_TEST_TRG_1 - Executed
TRIGGER_FOLLOWS_TEST_TRG_2 - Executed
```

```
1 row created.
```

```
SQL>
```

explain this

Great! Let's fully explain the concept and the example given about the **order of firing triggers** in Oracle:

---

## Order of Firing Trigger in Oracle

### The Core Idea

- Oracle allows you to create multiple triggers on the same table, for the same timing point (for example, you can have more than one **BEFORE INSERT** trigger on a table).
  - However, Oracle does **NOT** guarantee the order in which these triggers will **execute** unless you explicitly specify it (using the **FOLLOWS** clause in Oracle 11g and above).
- 

### Example Explanation

#### Step 1: Table Creation

```
sql

CREATE TABLE trigger_follows_test (
 id NUMBER,
 description VARCHAR2(50)
);
```

- A simple table called **trigger\_follows\_test** is created with two columns.
- 

#### Step 2: Two Triggers for the Same Event

```
sql

CREATE OR REPLACE TRIGGER trigger_follows_test_trg_1
BEFORE INSERT ON trigger_follows_test
FOR EACH ROW
BEGIN
 DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_1 - Executed');
END;
/
```

sql

```
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_2
BEFORE INSERT ON trigger_follows_test
FOR EACH ROW
BEGIN
 DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_2 - Executed');
END;
/
```

- Both triggers fire on the same event: BEFORE INSERT on trigger\_follows\_test.
- Each prints a different message when fired.

### Step 3: Insert Data

sql

```
SQL> SET SERVEROUTPUT ON
SQL> INSERT INTO trigger_follows_test VALUES (1, 'ONE');
```

- An insert is performed.
- Both triggers are executed, and their messages are printed.

### Output

sql

```
TRIGGER_FOLLOWS_TEST_TRG_1 - Executed
TRIGGER_FOLLOWS_TEST_TRG_2 - Executed
1 row created.
```

- Both triggers ran, but **Oracle does not guarantee which will run first.**
- In this example, TRG\_1 fired before TRG\_2, but this is **not guaranteed every time unless you use the FOLLOWS clause.**

## Key Point

Without the FOLLOWS clause, the execution order of triggers on the same event/timing point is undefined and can change.

If you want a specific order, you should write:

```
sql

CREATE OR REPLACE TRIGGER trigger_follows_test_trg_2
BEFORE INSERT ON trigger_follows_test
FOLLOWS trigger_follows_test_trg_1
FOR EACH ROW
...
```

This **guarantees** that `trigger_follows_test_trg_1` will fire first, then `trigger_follows_test_trg_2`.

## Summary Table

| Situation                                             | What happens?                                    |
|-------------------------------------------------------|--------------------------------------------------|
| Multiple triggers, same timing/event, no FOLLOWS used | Oracle does NOT guarantee firing order           |
| FOLLOWS clause used                                   | Oracle will execute them in your specified order |

## In Simple Words

- Order is random unless you tell Oracle explicitly.
- Use FOLLOWS to control the order of trigger execution.

Let me know if you want an example with the FOLLOWS clause!



#### 9.1.7.2 Compound Triggers

A compound trigger allows code for one or more timing points for a specific object to be combined into a single trigger. The individual timing points can share a single global declaration section, whose state is maintained for the lifetime of the statement. Once a statement ends, due to successful completion or an error, the trigger state is cleaned up. In previous releases this type of functionality was only possible by defining multiple triggers whose code and global variables were defined in a separate package, as shown in

73

Dr. Raihan Kamal

CHAPTER 9. DATABASE TRIGGERS

the Mutating Table Exceptions article, but the compound trigger allows for a much tidier solution.

The triggering actions are defined in the same way as any other DML trigger, with the addition of the COMPOUND TRIGGER clause. The main body of the trigger is made up of an optional global declaration section and one or more timing point sections, each of which may contain a local declaration section whose state is not maintained.

See the example in the web given at the beginning of the section.

#### 9.1.7.3 Enable and Disable Triggers

It has been possible to enable and disable triggers for some time using the ALTER TRIGGER and ALTER TABLE commands.

```
ALTER TRIGGER <trigger-name> DISABLE;
ALTER TRIGGER <trigger-name> ENABLE;
ALTER TABLE <table-name> DISABLE ALL TRIGGERS;
ALTER TABLE <table-name> ENABLE ALL TRIGGERS;
```

expalin this

Certainly! Let's break down and explain both **compound triggers** and how to **enable/disable triggers** based on your text.

## 9.1.7.2 Compound Triggers

### What is a Compound Trigger?

- A **compound trigger** lets you group code for multiple timing points (like BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, AFTER STATEMENT) for a table or view **into a single trigger object**.
- This means instead of creating multiple separate triggers for the same table (for example, one BEFORE EACH ROW and another AFTER STATEMENT), you can combine all related code in **one place**.

### How does it work?

- **Global Declaration Section:**

The trigger can have a section at the top where you declare variables. These variables are **shared across all timing points** in that trigger and keep their values throughout the whole statement execution.

- **Timing Point Sections:**

Inside the compound trigger, you write separate sections for each timing point

(e.g., **BEFORE STATEMENT**, **BEFORE EACH ROW**, **AFTER EACH ROW**, etc.).

- When the statement finishes (either successfully or with an error), Oracle **cleans up the state** (i.e., variables and resources).

## Why use a compound trigger?

- It helps solve problems like the **mutating table exception** (when a row-level trigger tries to query or modify the table that caused it to fire).
- It's cleaner and more maintainable than creating multiple separate triggers and possibly a package for shared variables.

## Summary Table:

| Feature          | Compound Trigger                      | Regular Trigger              |
|------------------|---------------------------------------|------------------------------|
| Timing points    | Multiple in one trigger               | One timing point per trigger |
| Shared variables | Yes (global section)                  | No (unless using a package)  |
| Use case         | Complex logic, mutating table, audits | Simple row/statement logic   |

## 9.1.7.3 Enable and Disable Triggers

### What is it about?

- **Enabling and disabling triggers** lets you temporarily stop or restart triggers without dropping or recreating them.
- Useful for bulk operations, data migrations, or maintenance when you don't want triggers to fire.

### How do you enable/disable triggers?

You use the **ALTER TRIGGER** or **ALTER TABLE** commands:

- **Disable a specific trigger:**

```
sql
ALTER TRIGGER <trigger-name> DISABLE;
```

- **Enable a specific trigger:**

```
sql
```

```
ALTER TRIGGER <trigger-name> ENABLE;
```

- **Disable all triggers on a table:**

```
sql
```

```
ALTER TABLE <table-name> DISABLE ALL TRIGGERS;
```

- **Enable all triggers on a table:**

```
sql
```

```
ALTER TABLE <table-name> ENABLE ALL TRIGGERS;
```

### Example Use:

- If you're loading millions of rows and don't want audit triggers to slow you down, you can temporarily disable them, load the data, and then enable them again.

---

## In Simple Words:

- **Compound Trigger:**

Write all your related trigger logic (for different timing points) in one place, and share data/variables easily.

- **Enable/Disable Triggers:**

Temporarily turn triggers off/on as needed using simple SQL commands, without dropping them.

Let me know if you want to see a sample compound trigger code or examples of when to enable/disable triggers!